



Asistente de Soporte con IA para Café Aroma

Integración de una app Android, un agente conversacional (IA) y un ERP Odoo 17

Proyecto:	Proyecto Fin de Ciclo — Memoria técnica
Ciclo formativo:	2º DAM (Desarrollo de Aplicaciones Multiplataforma) — Dual
Alumno/a:	[Nombre y apellidos del alumno/a]
Tutor/a docente:	[Nombre del tutor/a]
Tutor/a de empresa:	[Nombre del tutor/a de empresa, si aplica]
Centro educativo:	[Nombre del centro]
Curso académico:	2025 / 2026
Fecha de entrega:	27 de agosto de 2026

Índice

1. Introducción y motivación

2. Objetivos del proyecto

3. Estudio de mercado

4. Presupuesto

5. Análisis y diseño técnico

6. Herramientas y tecnologías empleadas

7. Implementación — funcionalidades desarrolladas

8. Pruebas y validaciones realizadas

9. Problemas encontrados y soluciones adoptadas

10. Conclusiones

11. Líneas futuras y ampliaciones

12. Bibliografía y webgrafía

13. Manual técnico / de instalación

14. Manual de usuario

15. Anexo — Declaración de conformidad de accesibilidad (Nivel A)

1. Introducción y motivación

1.1. Nombre del proyecto

Asistente de Soporte con IA para Café Aroma. El nombre resume las tres piezas del sistema: un *asistente* conversacional que atiende a los clientes, orientado a *soporte* (consultas, citas, incidencias) y construido sobre *inteligencia artificial* (un modelo de lenguaje que decide qué herramientas ejecutar). "Café Aroma" es el negocio ficticio usado como caso de estudio: una cafetería con clientes registrados, productos, pedidos, facturas y un sistema de reservas de mesa/cita.

1.2. Motivación

Los negocios pequeños y medianos (cafeterías, tiendas, talleres) atienden por teléfono o WhatsApp preguntas repetitivas: "¿tenéis mesa libre el sábado?", "¿cuál es el estado de mi pedido?", "quiero cambiar mi cita". Resolverlas consume tiempo del personal y depende del horario de apertura. La motivación del proyecto es demostrar, con un caso completo y funcional, que un agente de IA puede resolver ese tipo de consultas de forma autónoma y seis días a la semana, sin sustituir al ERP de gestión (Odoo) sino apoyándose en él como única fuente de verdad — el agente nunca inventa datos ni mantiene su propia base de datos de clientes, pedidos o citas.

Desde el punto de vista académico, el proyecto permite trabajar de forma integrada casi todos los módulos del ciclo de **DAM**: desarrollo de interfaces Android nativas (acceso a datos, programación multimedia y de dispositivos móviles), acceso a datos y programación de servicios y procesos (API REST en Python), sistemas de gestión empresarial (Odoo como ERP real), y despliegue de aplicaciones (Docker, contenedores, arquitecturas cliente-servidor).

1.3. Imagen corporativa

Se ha definido una identidad visual sencilla y coherente, aplicada tanto a la app Android como a esta documentación:

Elemento	Valor	Uso
Color principal (acento)	#EC3013	Botones de acción, títulos destacados, elementos interactivos
Variantes del acento	#FFF2EF · #DD2B0F · #AE1800 · #4D170E	Estados (hover/pressed), fondos suaves, texto sobre acento
Fondo	#F3F2F2	Fondo general de pantallas
Texto principal	#201E1D	Texto sobre fondo claro
Neutros	#DEDDDD · #6D6A6A · #514E4E	Bordes, texto secundario, iconografía inactiva

Tipografía	Segoe UI / Roboto (sistema)	Sin fuentes personalizadas, para minimizar peso de la app
------------	-----------------------------	---

El nombre visible de la aplicación es "**Asistente de soporte**", con un icono en tonos rojo-naranja que evoca la calidez de una cafetería sin depender de una marca real, ya que "Café Aroma" es un cliente de ejemplo.

2. Objetivos del proyecto

2.1. Objetivo general

Diseñar, implementar y validar un sistema de atención al cliente asistido por IA para un negocio de hostelería, integrado con un ERP real (Odoo), accesible desde una app Android nativa y desde el propio backoffice del negocio.

2.2. Objetivos específicos

1. Implementar un agente conversacional capaz de entender lenguaje natural en español y ejecutar acciones reales sobre Odoo (consultar y agendar), usando el *OpenAI Agents SDK* con un modelo de lenguaje (Gemini) como motor de razonamiento.
2. Garantizar que el agente **nunca pueda suplantar ni acceder a datos de otro cliente**: la identidad del usuario se resuelve en el servidor a partir de su sesión, nunca a partir de lo que el modelo de lenguaje "dice" en la conversación.
3. Permitir que cada cliente autentique con sus propias credenciales reales de Odoo (usuario de portal), en lugar de un usuario genérico o simulado.
4. Ofrecer un control de permisos granular por cliente (qué acciones puede pedirle al asistente: agendar citas, consultar facturas, consultar stock, etc.), configurable por el administrador desde el propio Odoo.
5. Delimitar cada conversación como una *consulta* con inicio y fin explícitos, y dejar un registro auditable de cada una en Odoo, visible para el personal del negocio.
6. Construir una interfaz Android cuidada, con componentes nativos de Material Design, feedback visual (indicador de "escribiendo...", animaciones de pulsación) y formularios embebidos dentro del propio chat para acciones estructuradas (p. ej. agendar una cita con selector de fecha/hora nativo).
7. Documentar y aplicar buenas prácticas de seguridad en la integración con el ERP: usuario de integración de solo lectura, con permisos de escritura excepcionales y mínimos, y separación estricta entre lo que el modelo de IA puede "pedir" y lo que el backend permite ejecutar.

3. Estudio de mercado

3.1. Contexto y problema

La atención al cliente en pequeños negocios de hostelería se apoya hoy, mayoritariamente, en canales manuales: llamadas telefónicas, WhatsApp Business o formularios de contacto sin automatizar. Esto genera dos problemas recurrentes: (a) el negocio solo puede responder en su horario de apertura y con el personal disponible, y (b) las consultas repetitivas (horarios, disponibilidad, estado de un pedido) consumen tiempo que podría dedicarse a tareas de mayor valor.

3.2. Análisis de soluciones existentes

Solución	Enfoque	Limitación frente a este proyecto
Chatbots de flujo cerrado (árboles de decisión / botones)	Reglas fijas, sin comprensión de lenguaje natural	No entienden peticiones formuladas libremente ni combinan varias intenciones en un mismo mensaje
Bots de WhatsApp Business API genéricos	Mensajería automatizada básica, plantillas	No suelen integrarse de forma nativa y bidireccional con el ERP (consultar stock/facturas reales, crear una cita reservando el calendario oficial)
Módulos de chat de Odoo (Livechat / Helpdesk)	Chat en vivo o tickets, dentro del propio Odoo	Pensados para atención humana o tickets, no para un agente autónomo con razonamiento de IA que decide qué acción ejecutar
Asistentes de IA genéricos (ChatGPT, Gemini app) conectados de forma superficial	Conversación libre, sin conexión real a un sistema de gestión	No tienen forma de leer/escribir datos reales del negocio (clientes, citas, facturas) de forma segura y auditable

El punto diferencial de este proyecto es la combinación de los tres elementos a la vez: **comprensión de lenguaje natural** (vía LLM) + **ejecución real y auditada sobre el ERP** (vía Odoo, no una base de datos paralela) + **control de acceso por cliente** (permisos configurables, sesión ligada a un usuario real de Odoo). La mayoría de soluciones del mercado cubren solo una o dos de estas tres piezas.

3.3. Usuarios potenciales

- **Negocios de hostelería y comercio de proximidad** (cafeterías, restaurantes, peluquerías, talleres) que ya usan o podrían usar Odoo como ERP y quieren automatizar la atención de primer nivel.

- **Clientes finales de esos negocios**, que ganan un canal disponible 24/7 para consultar su información y gestionar citas sin llamar por teléfono.
- **Integradores/consultoras de Odoo**, como posible extensión o add-on vendible a sus clientes existentes.

3.4. Viabilidad y diferenciación

Odoo cuenta con más de 60 000 empresas clientes y un ecosistema de integradores muy activo, lo que hace de un add-on de este tipo un producto potencialmente reutilizable en múltiples clientes con configuración mínima (basta con adaptar el nombre del negocio y las herramientas expuestas al agente). El uso de un modelo de lenguaje accesible por API (Gemini, con capa gratuita) en lugar de infraestructura de IA propia reduce drásticamente el coste de entrada frente a desarrollar un motor de comprensión de lenguaje natural desde cero.

4. Presupuesto

4.1. Estimación de horas de desarrollo

Bloque de trabajo	Horas estimadas
Análisis, diseño y planificación	20 h
Configuración de Odoo (Docker, add-on <code>assistant_agent</code> , modelos, vistas, permisos)	35 h
Backend del agente (FastAPI, integración OpenAI Agents SDK + Gemini, herramientas, sesiones)	45 h
App Android (pantallas, Retrofit, componentes de chat, formularios interactivos, ajustes)	55 h
Seguridad y control de permisos (grupos Odoo, contexto de identidad, checkboxes por cliente)	15 h
Pruebas manuales y corrección de incidencias	25 h
Documentación (memoria, manuales, diagramas)	15 h
Total	≈ 210 h

4.2. Estimación de coste (hipótesis de mercado)

Concepto	Cálculo	Importe
Desarrollo (perfil junior/dual, 18 €/h)	210 h × 18 €	3 780 €
Licencias de software	Odoo Community (gratuito), Android Studio (gratuito), Python/FastAPI (gratuito)	0 €
API del modelo de lenguaje (Gemini)	Capa gratuita durante desarrollo; en producción, según volumen	0–20 €/mes
Hosting (servidor del agente + Odoo en un VPS)	Estimado, VPS de gama media	≈ 15–25 €/mes
Coste de desarrollo (una vez)		≈ 3 780 €
Coste de mantenimiento mensual		≈ 15–45 €/mes

Cifras orientativas con fines académicos, calculadas para justificar la viabilidad económica del planteamiento, no una oferta comercial real.

5. Análisis y diseño técnico

5.1. Arquitectura general

El sistema sigue una arquitectura de tres capas claramente separadas: la **app Android** (cliente), el **agente de IA** (servidor intermedio con la lógica conversacional) y **Odoo** (ERP, única fuente de datos). La app nunca habla directamente con Odoo: todo pasa por el agente, que es quien decide —apoyado en el modelo de lenguaje— qué herramienta ejecutar y con qué identidad.

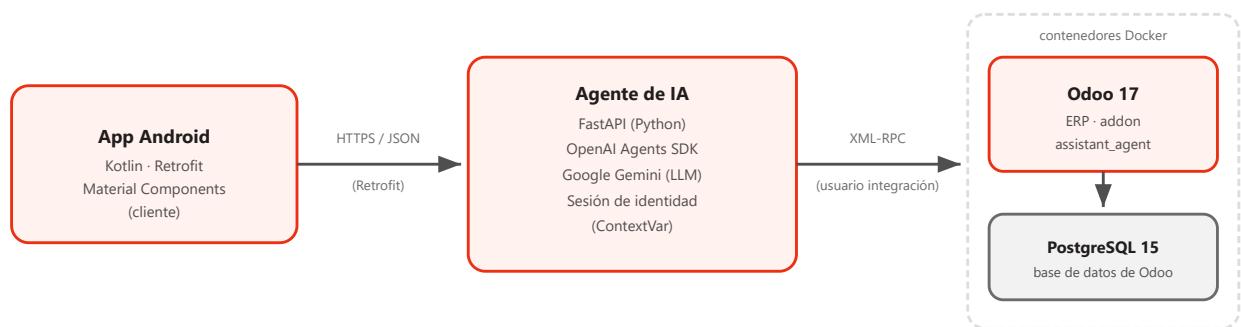


Fig. 1 — Diagrama de arquitectura / componentes del sistema.

5.2. Diagrama de casos de uso

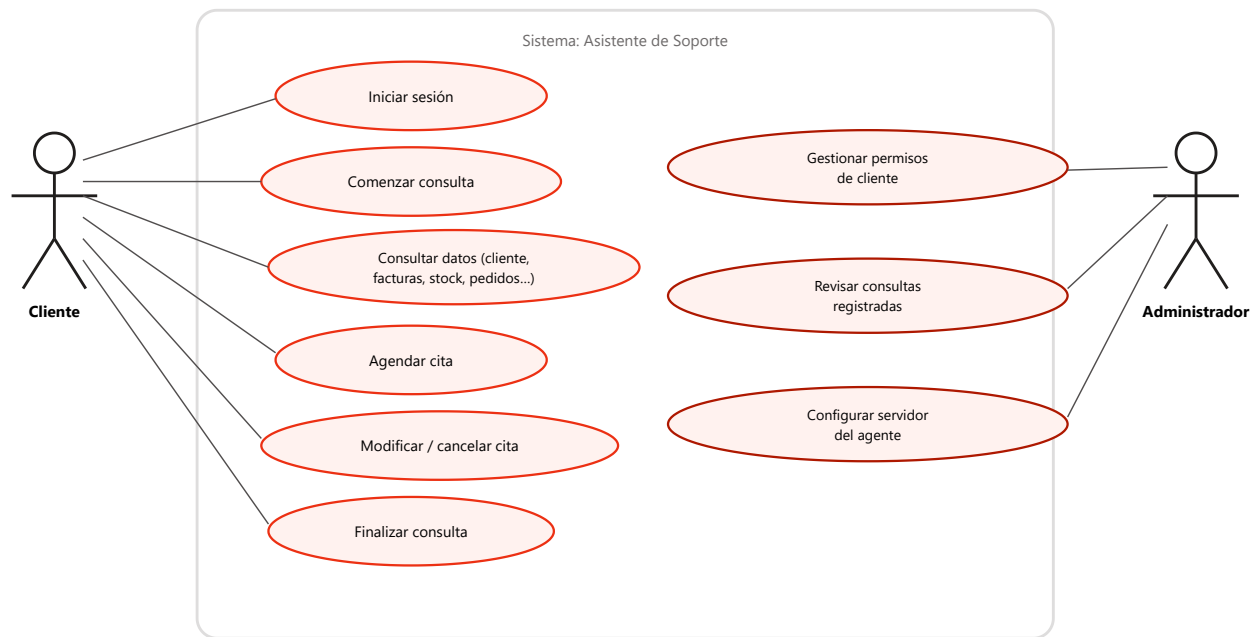


Fig. 2 — Diagrama de casos de uso (UML).

5.3. Modelo de datos (diagrama entidad-relación simplificado)

El agente no define un modelo de datos propio para la información de negocio: reutiliza los modelos estándar de Odoo (`res.partner`, `calendar.event`, `account.move`, `product.product`, `sale.order`, `stock.quant`) y añade únicamente dos piezas propias en el add-on `assistant_agent`: los campos de permisos sobre `res.partner` y el modelo nuevo `assistant.consulta`.

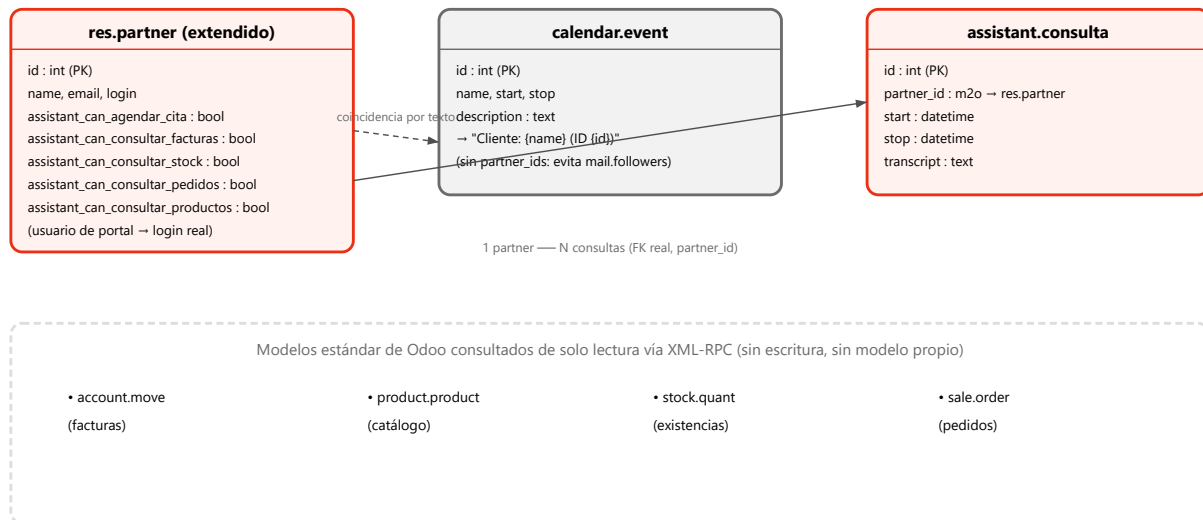


Fig. 3 — Modelo de datos simplificado (entidad-relación).

5.4. Seguridad e identidad

La regla de diseño más importante del proyecto es que **el modelo de lenguaje nunca decide con qué identidad se ejecuta una acción**. El flujo es:

1. El cliente inicia sesión con sus credenciales reales de Odoo (usuario de portal); el agente valida esas credenciales contra Odoo por XML-RPC y, si son correctas, genera un *token de sesión* propio, guardado en memoria del servidor (`auth_session.store`), asociado al `partner_id` real del cliente.
2. Cada petición de chat de la app manda ese token, nunca el `partner_id` directamente. El servidor resuelve el token a un `partner_id` y lo guarda en un `ContextVar` propio del hilo de la petición.
3. Las herramientas del agente (crear cita, consultar facturas, etc.) leen ese `ContextVar` para saber "quién pregunta" — el LLM puede pedir "consulta mis facturas", pero nunca puede pedir "consulta las facturas del cliente 42": no tiene ninguna forma de pasar un `partner_id` arbitrario a las herramientas.
4. Antes de ejecutar cada herramienta, se comprueba el campo de permiso correspondiente en `res.partner` (p. ej. `assistant_can_agendar_cita`); si el administrador lo ha desactivado para ese cliente, la herramienta rechaza la acción aunque el LLM la solicite.
5. El propio agente se conecta a Odoo con un **usuario de integración de solo lectura** (grupo de seguridad `group_agent_readonly`), con permisos de escritura habilitados de forma

explícita y mínima solo donde el negocio lo requiere (citas de calendario, registro de consultas).

5.5. Patrones de diseño y buenas prácticas aplicadas

Más allá de la arquitectura general, el código se apoya en varios patrones de diseño reconocidos, aplicados de forma consciente y no accidental:

Patrón	Dónde se aplica	Para qué
Repository	<code>AssistantRepository</code> (interfaz) con dos implementaciones — <code>RetrofitAssistantRepository</code> (real) y <code>MockAssistantRepository</code> (datos en memoria)	Las pantallas (fragments) piden datos a través de la interfaz, sin saber si vienen de la red o de un mock; permitió construir y probar la UI antes de que existiera el backend real, y sigue permitiendo simular partes que aún no tienen servidor (p. ej. <code>getCompanies()</code>)
Gateway / Facade	<code>agent/nodes/odoo_connector.py</code> : único punto del agente con permiso para hablar XML-RPC con Odoo	Concentra toda la complejidad de autenticación, reintentos y manejo de errores de Odoo en un solo módulo; el resto del agente solo ve funciones simples (<code>search_read</code> , <code>create_record</code> ...) y no puede saltarse esta frontera de seguridad
Adapter (RecyclerView)	<code>ChatAdapter</code> , <code>ConsultaAdapter</code>	Traduce una lista de datos del dominio (<code>ChatMessage</code> , <code>Consulta</code>) en filas visuales reutilizables, patrón estándar de Android
Inversión de dependencias por lambdas	Callbacks (<code>onClick</code> , <code>onPickDateTime</code> , <code>onSubmitForm</code>) inyectados por constructor en los adapters	El adapter no conoce el Fragment que lo usa ni cómo navega; solo invoca la función que le pasaron, reduciendo el acoplamiento

Ejemplo real de Repository + inyección de la implementación (`data/AssistantRepository.kt` y `data/RetrofitAssistantRepository.kt`):

```
interface AssistantRepository {
    suspend fun login(email: String, password: String): Result<String>
    suspend fun getConsultas(): Result<List<Consulta>>
    // ...
}

object RetrofitAssistantRepository : AssistantRepository {
    override suspend fun getConsultas(): Result<List<Consulta>> {
        val token = SessionManager.getToken() ?: return Result.failure(...)
        return try {
```

```
        val result =
AgentApiClient.api.listarConsultas(ListarConsultasRequestDto(token))
        Result.success(result.map { Consulta(it.start, it.stop, it.transcript,
it.state) })
    } catch (e: IOException) { Result.failure(...) }
    }
}

// En el Fragment: solo se conoce la interfaz, nunca la implementación concreta.
private val repository = RetrofitAssistantRepository // AssistantRepository
```

Buenas prácticas de programación aplicadas de forma transversal: separación estricta por paquetes (`data` , `model` , `ui` , `util` en Android; `nodes` , `config` en el agente), un único punto de acceso a Odoos (nunca SQL directo, ver §5.4), funciones cortas con un solo propósito, nombres descriptivos en español coherentes con el dominio del negocio, y comentarios que explican *por qué* se decidió algo (no *qué* hace una línea obvia).

6. Herramientas y tecnologías empleadas

Capa	Tecnología	Motivo de elección
App Android	Kotlin	Lenguaje oficial de Android, conciso y con soporte de corrutinas para llamadas asíncronas
	Retrofit2 + OkHttp3 + Gson	Cliente HTTP estándar de facto en Android para consumir APIs REST/JSON
	Material Components + Navigation Component	Componentes visuales accesibles y consistentes, navegación declarativa entre fragments
	ViewBinding	Acceso a vistas sin <code>findViewById</code> , con seguridad de tipos en compilación
	SharedPreferences	Persistencia ligera de ajustes (URL del servidor, sesión)
Agente de IA	Python 3 + FastAPI	Framework asíncrono, ligero, con validación automática de datos vía Pydantic
	OpenAI Agents SDK	Orquestación de agente + herramientas (tools) con detección de qué función se ha invocado
	Google Gemini (API compatible con OpenAI)	Modelo de lenguaje con capa gratuita suficiente para desarrollo y pruebas
	python-dotenv	Gestión de configuración/secretos por entorno (<code>.env</code>)
	FastAPI CLI	Arranque del servidor (<code>fastapi run</code> / <code>fastapi dev</code>), documentación interactiva automática en <code>/docs</code>
ERP / datos	Odoo 17 (Community)	ERP open-source con módulo de contactos, calendario, ventas, facturación y stock ya integrados
	XML-RPC	API externa oficial de Odoo, no requiere instalar dependencias adicionales en el servidor
	PostgreSQL 15	Motor de base de datos requerido por Odoo
Infraestructura	Docker + docker-compose	Entorno reproducible de Odoo + Postgres, aislado de la máquina de desarrollo
	Git	Control de versiones del código de los tres componentes

7. Implementación — funcionalidades desarrolladas

A continuación se resumen los bloques funcionales realmente construidos y verificados durante el proyecto, en el orden en que se fueron incorporando:

7.1. Rediseño de interfaz

Rediseño completo de la app sobre un sistema visual propio (colores, tipografía, componentes reutilizables): burbujas de chat con esquinas redondeadas, indicador de "escribiendo..." animado, transiciones de pantalla suaves y feedback de pulsación en botones, sustituyendo los símbolos de texto iniciales por iconografía Material.

7.2. Herramientas de negocio para Café Aroma

Conjunto de herramientas (*tools*) del agente para consultar información real del cliente en Odoo: datos de cliente, facturas, catálogo de productos, existencias en stock y pedidos — todas de solo lectura y con datos de demostración cargados en Odoo.

7.3. Permisos por cliente

Campos booleanos añadidos a `res.partner` (uno por cada acción del agente) gestionables por el administrador desde una vista de formulario de Odoo, comprobados por el agente antes de ejecutar cada herramienta.

7.4. Autenticación real de clientes

Sustitución del acceso simulado por autenticación contra usuarios reales de **Odoo Portal** (`base.group_portal`): el cliente introduce su email/contraseña reales y el servidor valida esas credenciales directamente contra Odoo.

7.5. Formulario interactivo de cita en el chat

Cuando el agente detecta la intención de agendar una cita, en lugar de pedir los datos por texto libre, la app reconoce la llamada a la herramienta correspondiente (`solicitar_datos_cita`, detectada de forma estructural a través de `ToolCallItem`, no analizando el texto) y renderiza dentro de la propia burbuja del chat una tarjeta con selector de fecha (`MaterialDatePicker`), selector de hora (`MaterialTimePicker`) y campos de motivo/duración.

7.6. Consultas reales con inicio y fin explícitos

Cada conversación pasa a ser una *consulta* delimitada: el cliente pulsa "Comenzar consulta" para abrir una conversación nueva (con su propio identificador e historial vacío) y "Finalizar consulta"

para cerrarla. Al finalizar, el servidor construye la transcripción completa del intercambio y crea un registro en el nuevo modelo `assistant.consulta` de Odoo (cliente, inicio, fin, transcripción), visible para el personal del negocio desde el propio backoffice de Odoo.

7.7. Historial de "Mis consultas" para el cliente

La pestaña "Consultas" de la app, que originalmente mostraba una simulación de tickets sin conexión real (un flujo de "proponer cambio → aprobar/rechazar" nunca conectado a nada), se sustituyó por el historial real de las consultas ya cerradas del propio cliente. Se añadió un nuevo endpoint (`POST /consulta/listar`) que devuelve, a partir del token de sesión, únicamente las consultas del cliente autenticado (filtrado por `partner_id` en el propio agente, igual que para facturas/pedidos/stock); tocar una fila abre la transcripción completa. El flujo antiguo de tickets simulados se conservó intacto donde seguía usándose desde otra parte de la app (el Panel y las tarjetas de "acción propuesta" del Chat), sin tocarlo.

7.8. Ajuste visual del botón "Finalizar consulta"

El botón para cerrar una consulta era un simple texto sin fondo (parecía una etiqueta, no un botón). Se sustituyó por un botón tonal compacto (`Button.Chip.Accent`: fondo `color_accent_100`, texto `color_accent_700`), coherente con el resto de componentes interactivos de la app.

7.9. Estado de consulta (pendiente/resuelta) y control exclusivo del agente desde Odoo

Se retiraron por completo de la app las pestañas "Panel" y "Trazas" y las tarjetas de "acción propuesta" del Chat (botón "Revisar" → aprobar/rechazar con traza de ejecución paso a paso): eran un flujo simulado de un sistema de aprobación que nunca llegó a conectarse a datos reales (la lista de tickets estuvo vacía toda la vida del proyecto). El criterio aplicado es que **el control del agente —qué se ha ejecutado, qué queda por revisar— es una responsabilidad exclusiva del administrador desde el propio Odoo**, no de la app del cliente.

A cambio, el modelo `assistant.consulta` ganó un campo `state` (selección: *pendiente/resuelta*, por defecto *pendiente* al cerrarse el chat). El admin cambia ese estado desde un widget de barra de estado en el formulario de Odoo; el bot de IA solo puede leerlo, nunca escribirlo (permiso de solo lectura sin escritura en `ir.model.access.csv`). La app, en la pestaña "Consultas", pasa a mostrar dos pestañas de filtro reales — Pendiente/Resuelta — sobre el historial ya construido, con una insignia de estado en cada fila y en el detalle; el cliente solo puede consultarlo, nunca cambiarlo.

8. Pruebas y validaciones realizadas

Dado que el proyecto integra tres sistemas independientes, cada funcionalidad se validó de extremo a extremo, no solo a nivel de código:

Tipo de prueba	Método	Ejemplo real realizado
Pruebas de API (backend)	<code>curl</code> contra los endpoints de FastAPI	Login → varios turnos de <code>/chat</code> con el mismo <code>conversation_id</code> → <code>/consulta/finalizar</code> , comprobando la respuesta JSON en cada paso
Verificación de datos	Consultas SQL directas sobre la base de datos de Odoo (Postgres)	Comprobar tras cada prueba que la tabla <code>assistant_consulta</code> contenía una fila nueva con el <code>partner_id</code> correcto y la transcripción esperada; limpieza de datos de prueba después de cada verificación
Pruebas de permisos	Desactivar un permiso desde Odoo y repetir la petición correspondiente	Con <code>assistant_can_agendar_cita</code> desactivado, comprobar que el agente rechaza agendar aunque el cliente lo pida explícitamente
Pruebas de aislamiento de identidad	Comprobar que ninguna herramienta acepta un identificador de cliente distinto al de la sesión activa	Verificar que las herramientas de consulta solo usan el <code>partner_id</code> resuelto por el servidor, nunca uno indicado en el texto del mensaje
Pruebas manuales de UI	Emulador Android (Medium Phone, API 36), interacción real con la app	Flujo completo: login real, comenzar consulta, rellenar el formulario de cita con el selector nativo, enviar, finalizar consulta, comprobar que vuelve a la pantalla de "sin consulta activa"
Pruebas de regresión visual	Comparación en distintos tamaños de pantalla / modo de navegación gestual	Detección y corrección del solapamiento de la barra de navegación del sistema con el contenido inferior de la app (edge-to-edge)
Pruebas de reinicio/persistencia	Reinicio manual de contenedores y del proceso del agente	Comprobar que un reinicio de Odoo requiere <code>button_immediate_upgrade</code> tras el reinicio del contenedor para modelos nuevos, y que un reinicio del agente invalida las sesiones en memoria (login obligatorio de nuevo)

Todas las incidencias detectadas durante estas pruebas se documentan en el apartado siguiente junto con su solución. Una de ellas —un botón de una lista que no abría el detalle al tocarlo— solo se detectó precisamente por probar la app real en el emulador y no fiarse únicamente de que el proyecto compilase.

8.1. Validación de código

Además de las pruebas funcionales de la tabla anterior, el código en sí se validó de forma explícita antes de darlo por bueno, con una herramienta distinta por cada lenguaje del proyecto:

Lenguaje / capa	Validador empleado	Qué comprueba
Kotlin (Android)	Compilador de Kotlin vía Gradle (<code>gradlew compileDebugKotlin</code> y <code>gradlew assembleDebug</code>)	Tipado estático, resolución de referencias, generación de <i>binding</i> de vistas; se ejecutó tras cada bloque de cambios, no solo al final, deteniéndose a corregir cualquier error antes de continuar
XML (layouts, navegación, menús)	Verificación de recursos de Android integrada en el propio <code>assembleDebug</code> (<code>processDebugResources</code>)	Que cada <code>@id</code> / <code>@string</code> / <code>@drawable</code> referenciado exista realmente y que el XML esté bien formado
Python (agente)	<code>ast.parse()</code> sobre cada fichero modificado, más una importación real de la app FastAPI (<code>from agent.main import app</code>) para forzar la resolución de todos los imports y decoradores de rutas	Sintaxis válida y que la aplicación arranque sin excepciones antes de exponerla por HTTP
XML de vistas de Odoo	El propio cargador de módulos de Odoo, al ejecutar <code>-u assistant_agent</code>	Que las vistas (<code>tree</code> , <code>form</code> , <code>header</code> / <code>statusbar</code>) sean XML válido y referencien campos que existen de verdad en el modelo — un error aquí detiene la actualización del módulo con traza clara

9. Problemas encontrados y soluciones adoptadas

Problema	Causa raíz	Solución adoptada
Odoo no reconocía modelos/vistas nuevos tras editarlos	Odoo cachea en memoria el registro de modelos y el manifiesto del módulo mientras el contenedor sigue vivo; los ficheros de datos nuevos no se detectan solo con <code>button_immediate_upgrade</code>	Reiniciar el contenedor (<code>docker restart odoo-odoo-1</code>) siempre que se añade un fichero Python o de datos nuevo al manifiesto, antes de actualizar el módulo
Error de permisos al crear citas de calendario vinculadas a un cliente (<code>message_follower_ids</code>)	El subsistema de "seguidores" de Odoo (<code>mail.thread</code>) restringe ese campo a nivel de núcleo a usuarios internos; no es corregible vía <code>ir.model.access.csv</code>	No usar <code>partner_ids</code> en <code>calendar.event</code> ; identificar al cliente como texto plano dentro de <code>description</code> ("Cliente: nombre (ID n)") y usar ese texto para búsquedas y comprobación de propiedad
Sucesión de errores de permisos al crear/leer citas ("acceso denegado" a distintos modelos)	Efectos colaterales de <code>calendar.event</code> sobre modelos auxiliares (<code>mail.activity</code> , <code>calendar.recurrence</code> , <code>res.users</code> , <code>mail.message.subtype</code> , etc.) sin permisos concedidos al usuario de integración	Añadir, uno a uno según aparecía el error, permisos de lectura mínimos y explícitos en <code>ir.model.access.csv</code> para cada modelo auxiliar necesario
El administrador no podía ver ni borrar los registros de <code>assistant.consulta</code>	Solo se había concedido permiso de creación al grupo del bot; se olvidó dar acceso al grupo de usuarios internos	Añadir una segunda línea en <code>ir.model.access.csv</code> con permisos completos para <code>base.group_user</code>
Contenido oculto tras la barra de navegación del sistema en varias pantallas	<code>targetSdkVersion 36</code> fuerza el modo <i>edge-to-edge</i> por defecto en Android, y el contenido no reservaba espacio para las barras del sistema	Función de extensión reutilizable sobre <code>WindowInsetsCompat</code> , aplicada al contenedor inferior de cada pantalla afectada (no a la raíz completa, para no desplazar el resto del contenido)
Errores 429 (límite de peticiones) del modelo de lenguaje durante pruebas intensivas	La capa gratuita de la API de Gemini limita a 20 peticiones/día para el modelo usado inicialmente	Cambio de modelo a una variante con límites más permisivos (<code>gemini-3.1-flash-lite</code>) mediante variable de entorno, sin cambios de código
Error 401 tras reiniciar el servidor del agente	Las sesiones de autenticación se guardan en memoria del proceso (decisión consciente de MVP); un reinicio del servidor las elimina todas	Documentado como limitación conocida; el usuario debe volver a iniciar sesión tras cada reinicio del agente. Recogido también

		como línea futura de mejora (persistencia de sesión)
Un permiso de lectura recién añadido en <code>ir.model.access.csv</code> seguía siendo rechazado por Odoo aunque la base de datos ya tenía el valor correcto	La actualización del módulo se lanzó con <code>odoo -u assistant_agent --stop-after-init</code> en un proceso aparte mientras el servidor principal seguía vivo; ese servidor mantiene en memoria una caché de permisos por grupo que no se invalidó con la actualización externa	Reiniciar el contenedor completo del servidor (<code>docker restart odoo-odoo-1</code>) para forzar una recarga limpia del registro y sus cachés de seguridad, no solo ejecutar la actualización del módulo
Al pulsar una fila del historial de consultas no pasaba nada (no abría el detalle)	El <code>RecyclerView.Adapter</code> ataba el listener de clic a la vista raíz del layout de la fila, pero el elemento hijo interno (<code>rowContent</code>) ya era <code>clickable</code> por su fondo de <i>ripple</i> — interceptaba el toque antes de que llegara a la raíz, sin tener ningún listener propio	Mover el <code>setOnClickListener</code> al propio <code>rowContent</code> , el elemento que realmente recibe el toque. Detectado y corregido en una verificación manual real sobre el emulador, no solo compilando
Fallo de Postgres ("could not serialize access due to concurrent update") al actualizar el módulo de Odoo tras un reinicio	Se lanzó <code>odoo -u assistant_agent --stop-after-init</code> demasiado pronto tras <code>docker restart</code> , mientras el propio servidor principal aún estaba terminando su propia carga de módulos en otra transacción	Esperar a que el log del contenedor confirme "Registry loaded" antes de lanzar cualquier actualización de módulo en un proceso aparte
Elemento visual extraño bajo la pantalla de login en el emulador	Confundido inicialmente con un fallo de la app; tras inspección con <code>dumpsys</code> , resultó ser en parte el "Taskbar" propio de Android (artefacto de la navegación gestual) y en parte el propio marco de la ventana del emulador de Android Studio	Distinguido de un fallo real de la app; solucionado el artefacto genuino de Android con un reinicio en frío del emulador (sin snapshot); el resto era interfaz del propio emulador, no de la aplicación

10. Conclusiones

El proyecto demuestra que es viable construir, con herramientas accesibles y sin infraestructura de IA propia, un asistente conversacional capaz de operar de forma segura sobre un ERP real. La parte más determinante del trabajo no ha sido conseguir que el modelo de lenguaje "entienda" al usuario —eso lo resuelve en gran medida el propio LLM— sino diseñar correctamente los límites de seguridad alrededor de él: qué puede pedir, con qué identidad se ejecuta cada acción y qué puede realmente escribir en el ERP. La experiencia con Odoo confirma que integrarse con un ERP de terceros mediante su API pública, aun siendo la vía recomendada y más estable, obliga a entender en profundidad subsistemas internos (como el de seguidores de `mail.thread`) que no están pensados para ser usados por integraciones externas restringidas.

A nivel personal/formativo, el proyecto ha permitido aplicar de forma conjunta contenidos de varios módulos del ciclo: desarrollo de interfaces móviles nativas, consumo y diseño de servicios REST, seguridad y control de acceso, y administración de un sistema de gestión empresarial real desplegado en contenedores.

11. Líneas futuras y ampliaciones

- **Persistencia de sesiones y de memoria de conversación** en una base de datos (hoy en memoria del proceso, se pierde al reiniciar el servidor).
- **Historial de consultas propio dentro de la app**, para que el cliente pueda revisar sus consultas pasadas sin depender del backoffice de Odoo.
- **Resumen automático de cada consulta** generado por el propio LLM, además de la transcripción completa.
- **Extensión a otros procesos de negocio** (tienda, logística/envíos) reutilizando el mismo patrón de herramientas + permisos ya construido.
- **Despliegue en producción** con HTTPS real, un proveedor de LLM con acuerdo de nivel de servicio y copias de seguridad automatizadas de Odoo.
- **Chips de comandos rápidos** en el chat (p. ej. "Ver mis pedidos", "Agendar cita") para reducir la escritura manual en consultas frecuentes.
- **Notificaciones push** en la app cuando el negocio confirma o modifica una cita agendada por el asistente.

12. Bibliografía y webgrafía

- Odoor S.A. — *Odoor 17.0 Documentation* (External API / XML-RPC, ORM, seguridad y grupos). <https://www.odoo.com/documentation/17.0/>
- OpenAI — *OpenAI Agents SDK Documentation* (agentes, herramientas, ejecución). <https://openai.github.io/openai-agents-python/>
- Google — *Gemini API Documentation* (endpoint compatible con OpenAI). <https://ai.google.dev/gemini-api/docs>
- Google — *Android Developers: Material Components, Navigation Component, ViewBinding, WindowInsetsCompat / edge-to-edge*. <https://developer.android.com/>
- Square — *Retrofit y OkHttp Documentation*. <https://square.github.io/retrofit/>
- FastAPI — *FastAPI Documentation* (Pydantic, FastAPI CLI). <https://fastapi.tiangolo.com/>
- Docker Inc. — *Docker Compose Documentation*. <https://docs.docker.com/compose/>
- Object Management Group — *UML Specification* (convenciones de diagramas de casos de uso y de clases). <https://www.omg.org/spec/UML/>

13. Manual técnico / de instalación

13.1. Requisitos previos

- Docker Desktop (para Odoo + PostgreSQL)
- Python 3.11+ y `pip` (para el agente)
- Android Studio (para compilar/ejecutar la app)
- Una clave de API de Google Gemini (capa gratuita válida)

13.2. Puesta en marcha de Odoo

```
cd odoo
docker compose up -d
# Primer arranque: crear la base de datos "odoo_test" desde
# http://localhost:8069 e instalar el módulo "assistant_agent".
# Tras cualquier fichero Python/datos nuevo en el addon:
docker restart odoo-odoo-1
# y volver a pulsar "Actualizar" (upgrade) sobre el módulo.
```

13.3. Puesta en marcha del agente

```
cd agent
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
# Configurar agent/.env: LLM_API_KEY, LLM_MODEL, LLM_BASE_URL,
# ODOO_URL, ODOO_DB, ODOO_USERNAME, ODOO_API_KEY
fastapi dev main.py --host 0.0.0.0 --port 8000
```

13.4. Puesta en marcha de la app Android

1. Abrir la carpeta `kotlin-app` en Android Studio.
2. Ejecutar en un emulador o dispositivo físico.
3. En la pantalla de login, entrar en "Ajustes del servidor" e indicar la URL del agente (por defecto, `http://10.0.2.2:8000/` desde el emulador, que apunta al `localhost` de la máquina anfitriona).
4. Iniciar sesión con un usuario de portal real de Odoo (email y contraseña).

13.5. Notas operativas

Las sesiones de login y el historial de conversación viven en memoria del proceso del agente: reiniciar el servidor (`fastapi run` / `fastapi dev`) obliga a todos los clientes a iniciar sesión de nuevo.

14. Manual de usuario

14.1. Acceso

Al abrir la app, el cliente introduce su email y contraseña de cliente (los mismos que usaría en el portal web de Café Aroma). Desde la pantalla de acceso puede entrar también en "Ajustes del servidor" si necesita cambiar la dirección del asistente.

14.2. Realizar una consulta

1. En la pestaña *Chat*, pulsar "**Comenzar consulta**".
2. Escribir la petición en lenguaje natural (por ejemplo: "quiero agendar una mesa para el sábado a las 20:00" o "¿cuál es el estado de mi último pedido?").
3. Si la petición requiere datos estructurados (como agendar una cita), el asistente muestra dentro del propio chat una tarjeta con calendario y campos para completar o confirmar los datos.
4. Al terminar, pulsar "**Finalizar consulta**". La conversación queda registrada y la app vuelve a la pantalla de inicio, lista para una consulta nueva y totalmente independiente.

14.3. Para el personal del negocio (administrador)

- Desde el backoffice de Odoo, en la ficha de cada cliente, se puede activar o desactivar qué acciones tiene permitido pedirle al asistente.
- El histórico de consultas cerradas (con su transcripción completa) es visible desde el menú correspondiente del módulo `assistant_agent` en Odoo.

15. Anexo — Declaración de conformidad de accesibilidad (Nivel A)

Esta declaración cubre **todas las pantallas y funcionalidades** de la app (login, ajustes, chat, formulario de agendar cita, listado y detalle de consultas). Se basa en una revisión manual del código y de la interfaz frente a los criterios de Nivel A de las WCAG 2.1 aplicables a una app móvil nativa — no en una auditoría automatizada certificada por un tercero, algo que sí queda como línea de mejora futura (junto con extender la conformidad a Nivel AA) antes de un hipotético despliegue en producción.

Criterio (WCAG 2.1, Nivel A)	Cómo se cumple en la app
1.1.1 Contenido no textual	Los iconos puramente decorativos (p. ej. la flecha de cada fila de consultas) llevan <code>contentDescription="@null"</code> a propósito, porque la fila ya tiene su propio texto; los iconos con función propia (botón enviar del chat) tienen descripción textual (<code>chat_send_content_description</code>); el avatar del usuario muestra sus iniciales como texto real, no como imagen.
1.3.1 Información y relaciones	Uso exclusivo de componentes nativos de Android (<code>TextView</code> , <code>EditText</code> , <code>Button</code> , <code>RecyclerView</code>) en vez de vistas genéricas dibujadas a mano, para que TalkBack anuncie automáticamente el rol de cada elemento.
1.3.2 Secuencia con significado	El orden de los elementos en cada layout XML coincide con el orden de lectura/foco esperado; no se reordena nada solo visualmente (sin <code>z-index</code> /superposiciones que rompan el orden lógico).
1.4.1 Uso del color	El estado de una consulta nunca se indica solo por color: la insignia siempre lleva el texto "Pendiente"/"Resuelta" además del color de fondo.
2.4.2 Pantallas con título	Cada pantalla muestra un título visible propio en la cabecera (<code>headerTitle</code> : "Conversación", "Consultas", "Consulta cerrada"...) y cada <code>Activity</code> tiene su etiqueta en el manifiesto.
2.4.3 Orden del foco	En el formulario de agendar cita el foco avanza en orden lógico: motivo → fecha/hora → duración → botón agendar, sin saltos artificiales.
3.1.1 Idioma de la interfaz	Toda la app está en español de forma consistente, sin mezclar idiomas dentro de una misma pantalla.
3.2.1 / 3.2.2 Al recibir el foco / al modificar datos	Ningún control cambia de pantalla o de contexto automáticamente al enfocarse o al escribir; los cambios de pantalla siempre los dispara una acción explícita del usuario (pulsar un botón).
3.3.1 Identificación de errores	Los errores (login incorrecto, fallo de conexión con el agente) se muestran como texto plano y comprensible bajo el formulario correspondiente, nunca solo con un color o un icono.
3.3.2 Etiquetas o instrucciones	Todos los campos de entrada tienen <code>hint</code> /etiqueta visible (Email, Contraseña, Motivo, Fecha y hora, Duración...), nunca solo un placeholder que desaparece al escribir.

4.1.2 Nombre, función, valor	Al no usar vistas personalizadas para los controles interactivos, cada uno expone automáticamente a los servicios de accesibilidad su rol (botón, campo de texto...) y su estado, sin necesidad de reimplementarlo a mano.
------------------------------	--

No se declara conformidad frente a los criterios de Nivel A específicos de navegación web (2.4.1 "Evitar bloques", 4.1.1 "Procesamiento" de HTML) por no ser aplicables a una app nativa sin contenido web embebido.